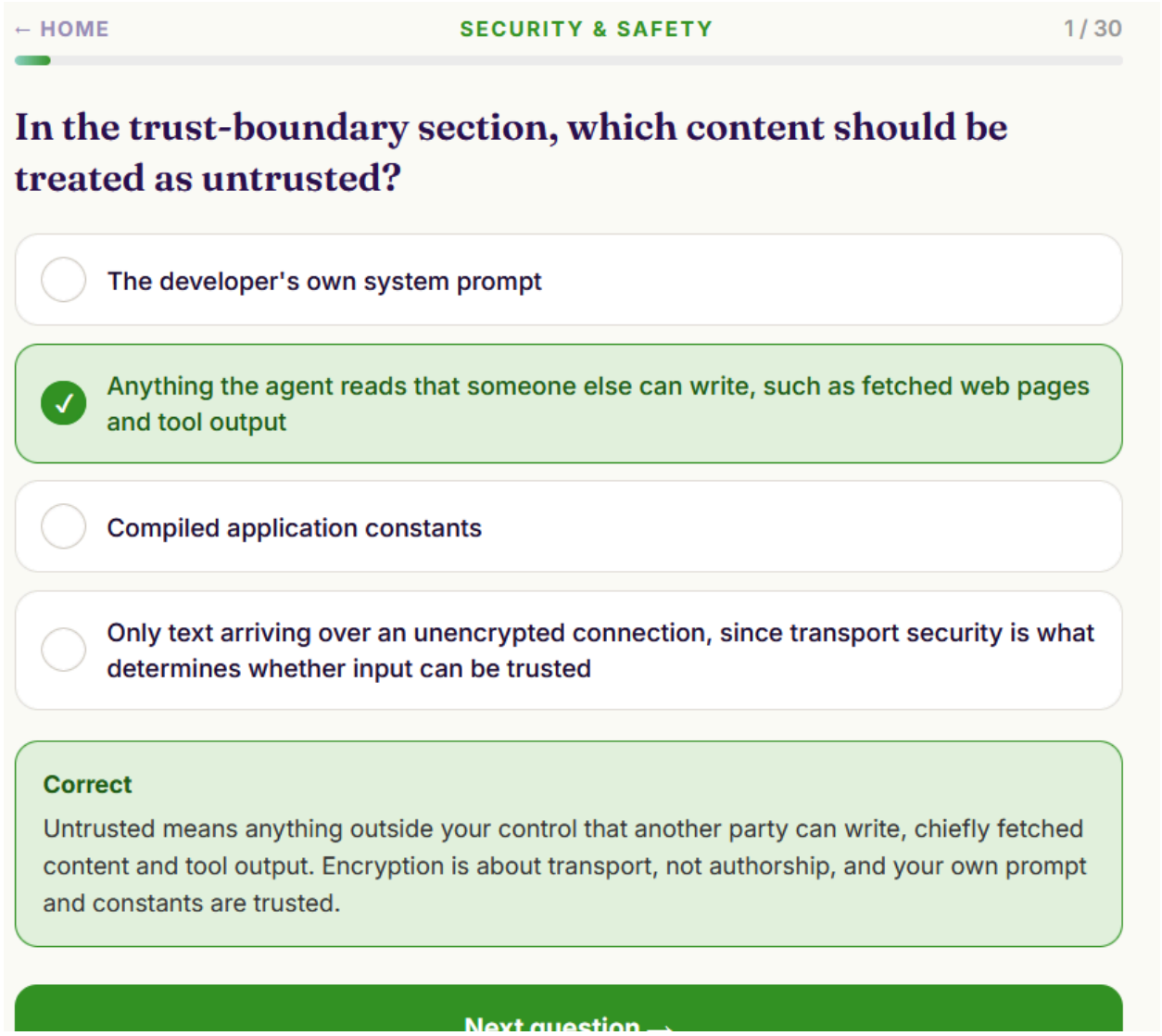
Set 2 : https://claude.ai/public/artifacts/83eee27a-0509-45c1-be08-cc3e8b4cbfc8



Your team already runs everything on AWS and needs Claude in the same account for governance. How should that shape platform choice?

- ✓ Use Amazon Bedrock so Claude runs within the existing AWS governance and data boundary
- Self-host only, as that is the sole compliant option
- Pick whichever platform shipped the newest model
- Use the first-party API and rebuild your governance tooling around it, since staying on one vendor's native platform is worth the migration regardless of where your data lives today

Correct

When governance and data residency point to an existing cloud, run Claude through that cloud's platform (here, Bedrock). Rebuilding tooling elsewhere, assuming only self-hosting is compliant, or chasing model recency all miss the constraint.

A one-off image is analysed once; a logo is reused across thousands of requests. How should each be supplied?

- ✓ Inline the one-off as base64, and put the reused logo behind the Files API so it isn't re-uploaded each call
- Batch both regardless of latency needs
- Inline both as base64 every time, since inlining is simplest and the extra bytes on the reused asset are negligible at scale
- Describe both in text to avoid image tokens

Correct

Inline base64 suits one-off images; the Files API suits assets reused across requests so they aren't re-sent each time. Inlining a reused asset repeatedly wastes tokens, and text description or forced batching miss the point.

Next question →

A regression test flakes because it compares the model's output to a fixed expected string. Best fix?

- Move the test to the largest model tier
- ✓ Assert on the structure or key facts of the output rather than an exact string, since generation is non-deterministic
- Pin temperature to 0 and assume output is now byte-identical every run, keeping the exact-string assertion in place
- Retry the test until it happens to pass

Correct

Generation is non-deterministic, so grade on structure or key facts, not an exact string. Temperature 0 reduces variation but does not guarantee identical bytes, and retrying or upsizing does not make the assertion valid.

Next question →

You must guarantee an agent can neither run rm nor read secrets/config.json, whatever it is asked. Which TWO settings pieces enforce this? (Pick 2)

Select 2 · chosen 2

- [ ] allow ["Bash(*)"]
- [x] deny ["Read(secrets/config.json)"]
- [x] deny ["Bash(rm:*)"]
- [ ] defaultMode "bypassPermissions"

Correct

Explicit deny rules at the settings layer block the destructive command and the secret read regardless of the session. bypassPermissions removes guards, and an unrestricted Bash allow does the opposite.

A plugin's SKILL.md hardcodes a tool at /Users/dev/tools/lint.sh and fails for teammates. Best fix?

- Delete the step so the skill no longer calls the script
- ✓ Reference it from the project root via CLAUDE_PROJECT_DIR
- Point every teammate's machine at a shared network mount at that same absolute path and document the mount in the README so setups stay consistent
- Use a home-directory shortcut like ~/tools/lint.sh

Correct

The defect is the machine-specific absolute path; CLAUDE_PROJECT_DIR resolves from the repo root anywhere. A shared mount or home shortcut just relocates the same fragility, and deleting the step drops the capability.

A product team wants one Skill to run inside a long-running agent that Anthropic hosts, reachable by an agent ID across sessions. What's required?

- ✓ Define the agent as an API resource that lists the skill and set the managed-agents beta header, writing the skill to avoid local-file dependencies since it runs in Anthropic's sandbox
- Set settingSources explicitly
- Send only the code-execution header
- Place SKILL.md in .claude/skills

Correct

An Anthropic-hosted agent addressed by ID is defined as an API resource listing the skill, with the managed-agents beta header, and the skill must not depend on local files. The others are the terminal and SDK runtimes.

Which TWO of these are gradeable success criteria you could build an eval against? (Pick 2)

Select 2 · chosen 2

- [ ] Handle the ticket well
- [x] Output valid JSON with fields sentiment (positive/negative/neutral) and confidence (0-1)
- [x] Classify the ticket into exactly one of five named categories
- [ ] Produce a genuinely useful triage

Correct

A constrained JSON schema and a one-of-five classification are checkable. 'Useful' and 'handle it well' are too vague to grade against.

Where do durable role and safety rules hold most reliably against later user turns?

- A tool result
- Repeated verbatim inside every assistant response, so the model is continually reminded of them throughout the conversation
- ✓ The system prompt
- The final user message

Correct

The system prompt is the authoritative, stable home for durable rules. Repeating them in assistant turns is fragile and wasteful, and user or tool content is lower priority.

You can write out the exact fixed sequence of steps a task always follows. Which architecture is right, and why?

- An agent, because agents are more capable and future-proof and you can always constrain it later if the extra latency and cost become a problem in production
- Independent, unordered calls
- ✓ A workflow, because the path is known and coding it avoids the cost and nondeterminism of an agent
- A single mega-prompt containing all steps

Correct

A known, fixed path is a workflow: deterministic and cheaper. Reaching for an agent adds nondeterminism and cost you do not need, and a mega-prompt or unordered calls cannot reliably sequence the work.

Shipping a reusable accelerator that must survive model updates. Which TWO practices matter most? (Pick 2)

Select 2 · chosen 2

- [x] Version prompts so changes are attributable and reversible
- [ ] Keep prompts inline and unversioned to stay lightweight
- [x] Pin the model version and upgrade deliberately after evals
- [ ] Auto-adopt each new model on release to stay current

Correct

Pinning the model and versioning prompts make change deliberate and reversible, which is what surviving updates requires. Auto-adopting and unversioned inline prompts remove exactly that control.

In the cost-and-latency section of a design doc, what is the 'reliability floor'?

- The cheapest model available
- ✓ The minimum reliability the design must hold and cannot trade away for cost or speed
- The lowest cost reachable if you are willing to accept as many dropped requests and timeouts as that price requires
- The p99 latency target

Correct

The reliability floor is the minimum reliability you refuse to trade for cost or latency. It is not a cost figure, a model choice, or a latency percentile.

Which TWO of these consume the context budget as a conversation grows? (Pick 2)

Select 2 · chosen 2

- [ ] The temperature setting
- [x] Prior turns kept in history
- [ ] The model's parameter count
- [x] Accumulated tool results

Correct

History and tool outputs both accumulate in the window and spend the fixed budget. Parameter count and temperature are model settings, not context the window holds.

A page fetched by a summariser contains hidden text telling the agent to email a file to an external address. Most effective mitigation?

- Switch to a larger model
- Add a system-prompt line telling the model to ignore instructions embedded in fetched pages
- Scan fetched pages for the word 'ignore'
- ✓ Least privilege: the summariser has no email capability, so injected instructions can't reach a send action, and untrusted content is kept separate from instructions

Correct

Least privilege means the tool simply cannot perform the injected action, which is the durable defense alongside isolating untrusted content. A prompt instruction is bypassable, keyword scanning is brittle, and a bigger model can be more compliant.

You point Claude Code at an unfamiliar third-party repo you don't fully trust. What permission posture fits the first pass?

- Disable all tools
- bypassPermissions, so exploration is fast and uninterrupted, on the reasoning that you'll review everything in the final diff before merging anyway
- Approve every file read manually
- ✓ Read-only plan mode to explore and propose before any edits

Correct

Match authority to risk: on an untrusted repo, plan mode explores read-only before changing anything. bypassPermissions grants far too much too early, approving every read is needless friction, and disabling tools defeats exploration.

An agent that can issue refunds is going to production, and refunds are irreversible. Correct design choice?

- A low temperature to reduce risky behaviour
- ✓ A human approval gate before the refund executes, wired in at design time
- A polite system-prompt reminder to be careful
- Detailed logging and alerting, so if a wrong refund goes out the on-call engineer is notified within minutes and can begin the reversal

Correct

Irreversible actions get a human gate before execution, designed in. Logging and alerting are after the money moved, and temperature or a prompt reminder are not enforceable controls.

Next question →

Your parser occasionally breaks on output that is valid JSON but includes a prose preamble. Most robust production handling?

- Trust the output and parse it directly
- Increase max_tokens
- ✓ Constrain to JSON-only and validate against a schema, repairing or rejecting malformed output before it flows downstream
- Set temperature to 0, which removes formatting variation entirely so a preamble can never appear again

Correct

Constrain the output and validate defensively before it moves on. Temperature 0 reduces but does not eliminate stray prose, trusting the output is what breaks, and max_tokens is unrelated.

A local SQLite inspection tool you use only on your own machine. Which transport and scope fit?

- ✓ stdio + Local
- HTTP + Project (.mcp.json)
- HTTP + Enterprise via managed settings, so it is centrally governed and consistently available to you across every environment you might ever work in
- HTTP + Local

Correct

A personal, same-machine tool uses stdio at local scope. Enterprise managed settings are for org-wide mandates, and project scope shares with a team, neither of which this needs.

Next question →

A committed credentials file exposed an API key. Two things are true: the key is being rejected, and it's stored in plaintext. Correct fix?

- Rotate the key and re-commit the new value to the same file so the app keeps working with no code changes, then restrict who can read the repo
- Add a retry with backoff so a later attempt succeeds
- Switch the service to OAuth
- ✓ Rotate the key and move it out of the file into an environment variable referenced at runtime

Correct

Both problems must be fixed: rotate the rejected key and stop storing it in plaintext by moving it to an environment variable. Re-committing leaves the storage defect, OAuth does not fit a service credential, and backoff does not fix a rejected key.

User-submitted text is concatenated straight into your prompt template and sometimes overrides your instructions. Which TWO help at the prompt layer? (Pick 2)

Select 2 · chosen 2

- [x] Keep trusted instructions in the system prompt, separate from user content
- [ ] Ask users nicely not to include instructions
- [ ] Truncate all user input to a fixed short length
- [x] Delimit and label the user text clearly as data

Correct

Delimiting user input as data and separating trusted instructions into the system prompt both blunt injection. A polite request is unenforceable, and blanket truncation mangles legitimate input.

Next question →

Writing the failure-handling section of a design doc, what must you decide for each error?

- Whether it is worth logging at all
- ✓ Whether it is retriable or terminal, and what the user gets when it can't be recovered
- The exact library and language for the retry, so the section is concrete enough for an engineer to build from without any further design work
- Which larger model tier to fail over to

Correct

Failure handling classifies each error retriable or terminal and defines the user-facing fallback. Naming implementation libraries is too low-level for the decision, and logging or tier-swaps are not the error path.

Next question →

An agent calls a search tool far more often than a create tool, even when creation is intended. Their descriptions overlap. Best first fix?

- ✓ Rewrite the descriptions so each states its distinct purpose and when not to use it
- Raise the temperature
- Remove the search tool
- Reorder the tools so the create tool is listed first, since the model tends to prefer whichever tool appears earlier in the list

Correct

Overlapping descriptions cause wrong-tool selection, so make each description distinct with an exclusion condition. List order is not the lever, removing a tool drops a capability, and temperature does not help.

Next question

A service calls the Messages API and wants a Skill to run as part of the request. What must be configured?

- Set settingSources explicitly for the Agent SDK
- Set the managed-agents beta header and list the skill on an agent resource
- ✓ Send the code-execution and skills beta headers, and write the skill so its steps don't depend on local files or tools
- Place SKILL.md in .claude/skills and let the terminal pick it up

Correct

For a Messages API request, send the code-execution and skills beta headers and keep the skill free of local-file dependencies. The terminal placement, settingSources, and managed-agents header belong to other runtimes.

Next question

Over a long conversation the model's output format slowly drifts from what you asked. Which technique addresses this failure type?

- ✓ Complete and tighten the system prompt so the format rule is stated durably
- Add three few-shot examples of the desired reasoning steps to every user message so the model re-anchors on structure each turn
- Switch models partway through
- Raise the temperature

Correct

Drift across turns points to an underspecified system prompt, so fix it there. Piling reasoning examples into every user turn is costly and off-target, and temperature or model swaps do not address drift.

A hook must prevent a tool from writing outside the project directory. Which TWO are correct? (Pick 2)

Select 2 · chosen 2

- [ ] Use PostToolUse and roll the write back afterwards
- [x] Exit with code 2 and put the reason on stderr to block it
- [x] Use the PreToolUse event, since it fires before the write executes
- [ ] Exit 0 with the reason on stdout

Correct

Only PreToolUse can block, and the hook signals a block with exit code 2 plus a reason on stderr. PostToolUse runs after the write, and exit 0 allows the call.

Next question →

A research agent must read 40 documents and produce one cited synthesis, but everything in one context degrades quality. Best approach?

- Lower the temperature
- ✓ Have subagents extract from each document in isolation, then synthesise from the compact extracts
- Load all 40 at once and raise max_tokens as high as the model allows so nothing has to be left out of the single context
- Use only the documents that fit comfortably

Correct

Isolating each document in a subagent keeps contexts clean and feeds synthesis compact extracts. Cramming all 40 in is the bloat that degraded quality, dropping documents loses data, and temperature is irrelevant.

Next question →

An agent's answer is wrong. The trace shows the tool returned correct data. Where is the failure?

- ✗ In the integration layer, because whenever a final answer is wrong the fault lies in how the tool output was parsed and handed back to the model
- In the API key configuration
- ✓ In the model's use of the data, not the tool
- In the network

Not quite

The tool returned correct data, so the fault is in how the model reasoned over it. The sweeping claim about the integration layer is false, and network and key issues would not produce correct tool data plus a wrong answer.

A workload is simple and latency-sensitive and does not need step-by-step reasoning. Which setting fits?

- ✓ Skip extended thinking, enabling it only where a reasoning pass changes the answer
- Turn on maximum-effort extended thinking so even the simple answers are extra reliable and you never regret it later
- Always route to the largest model
- Pin adaptive thinking to its highest effort permanently

Correct

Reasoning effort should be spent only where it changes the answer. Maxing thinking or the model tier on simple, latency-sensitive work just adds cost and latency.

Next question

In a long agent session the same instruction prefix is re-sent every turn. What reduces the repeated input cost?

- Adding more few-shot examples
- Lowering max_tokens each turn so responses are shorter, which brings the per-turn input cost down as the session goes on
- ✓ Cache checkpoints on the stable prefix
- Switching the session to batch mode

Correct

Cache checkpoints let the unchanged prefix be reused at reduced cost. max_tokens caps output not input, batch does not suit a live session, and more examples add tokens.